Date: November 12, 2025

How to Use the IMS Servers

This file lists many example websites, but there are thousands of other websites available that explain these steps in detail, feel free to use another one.

Overview

- GPU servers for students: nandu, strauss, kiwi
  - more info on schwarzmilan in /etc/nis-imsv/hosts
  - nandu is the only one with 48GB GPUs
- You can find more about other linux commands here: https://www.usm.uni-muenchen.de/people/puls/lessons/intro_general/Linux/Linux_for_beginners.pdf
- VPN access (not required to log into servers from outside the IMS):
  - Follow the instruction in this page https://www.tik.uni-stuttgart.de/en/services-a-z/VPN/

Server Access

- Create an ssh key and store it in your home directory (no matter which server you log into, you always have access to the same file system) https://www.digitalocean.com/community/tutorials/how-to-configure-ssh-key-based-authentication-on-a-linux-server
  - Nice to have for easiest access: create an ssh-config file locally and add the server there https://linuxize.com/post/using-the-ssh-config-file/
  - Alternatively to the ssh config, you can add an alias line into your .bashrc file: `alias ssh-connect='ssh -XL localhost:8888:localhost:8888 username@elsterweihe.ims.uni-stuttgart.de'`

Storage and Quota

- If you need more space, do one of the following things:
  - Ask admins for more space (preferred)
    - `/mount/studenten/…`
    - For student theses, admins usually create a directory for the student under one of the `arbeitsdaten-studenten` directories
    - If multiple students work on the same project, one can also ask for a project directory.
  - Ask your supervisor to get you access to a personal folder in research group directories (if you need access to data from there or so):
    - `/mount/projekte/…/<your_handle>`: for code, results, etc. (anything that can not easily be recreated)
    - `/mount/arbeitsdaten33/…/<your_handle>`: for environments etc. (that can easily be recreated and takes up lots of space)
  - Extra resource: `/mount/studenten-temp1/users`

        - You can create your own directory under here with your username. You don't need to ask for permission for this. No back-up. The data will be removed later on.
- Your home directory has a relatively small quota and you should not store big files that can easily be recreated in the `projekte` folder, here is how to avoid exceeding it
    - Check quota with `quota -v`
    - Set symlinks to your `arbeitsdaten` directory (the one with plenty of space) https://askubuntu.com/questions/843740/how-to-create-a-symbolic-link-in-a-linux-directory
        - … for your vs code cache
        - … for the huggingface cache folder in your .cache home directory
        - … for your pip or conda cache (packages from virtual environments)
    - Make sure to delete models you do not need anymore since they require a lot of disk space
    - Alternatively, you can redirect the hf cache by using environment variables

Setup

- Use virtual environments (https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/ or https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html)
    - Install them in the `arbeitsdaten33-directory` that was made for you
    - Alternatively, redirect your pip/conda cache (see above)
    - Always specify the python version when creating them
    - From time to time, export them into a `requirements.txt` or `environment.yml` file
        - Don't forget to activate them before running python scripts
- Set up a git repository with your code and clone it on the IMS server
    - You can use the github version provided by TIK (https://www.tik.uni-stuttgart.de/dienste-a-z/Git-Hosting/) or any other development platform
    - Not a hard requirement but strongly encouraged for the thesis
    - Always commit and push your code after completing a task
- Copy files to and from the server to your local machine:
    - scp: https://linuxize.com/post/how-to-use-scp-command-to-securely-transfer-files/
    - Alternatively, you can use rsync to sync (clone) your server directory to your local directory (https://www.digitalocean.com/community/tutorials/how-to-use-rsync-to-sync-local-and-remote-directories)
    - For your code, consider using git (see above)
- Connect your local VSCode to the server (https://code.visualstudio.com/docs/remote/ssh)

    - You can also create a symlink here for the .vscode-server folder in your home directory to your `arbeitsdaten33` directory to save space in the home directory
- In case you need to run a docker image:
    - In the server podman is installed instead of docker
    - To use docker, you have to edit to storage.conf:

```
[storage]
driver = "overlay"
runroot =
"/fs/scratch/users/[username]/podman/containers"
graphroot =
"/fs/scratch/users/[username]/podman/images"
[storage.options.overlay]
force_mask = "700"
```

Running Code

- You cannot log out of the server or close your computer when running long-running jobs directly from your terminal that is connected to the remote server
    - Use screen sessions (recommended) (https://linuxize.com/post/how-to-use-linux-screen/)
    - Alternatively, use tmux (https://www.redhat.com/en/blog/introduction-tmux-linux)
- When using screen, it makes sense to execute your python code from shell scripts
    - Use `fire` (or any other available package such as `click` or `argparse`) to pass parameters to your function in the shell script: https://github.com/google/python-fire
    - How to create and execute a shell script: https://stackoverflow.com/questions/4377109/shell-script-execute-a-python-program-from-within-a-shell-script

Using the GPUs

- When running python code, use the `CUDA_VISIBLE_DEVICES` parameter to choose one or more free GPUs
    - E.g. `CUDA_VISIBLE_DEVICES=2,3` python main.py will run the main.py scripts on the third and fourth GPU (zero-indexed)
    - You can find free GPUs with nvidia-smi
    - To have more control, you can set it up in your python code:
      import os
      `os.environ['CUDA_DEVICE_ORDER'] = 'PCI_BUS_ID' #this line is important`
      `os.environ['CUDA_VISIBLE_DEVICES'] = '2,3'`
- Currently, you might experience issues when trying to run models on two or more GPUs
    - Either it works, or you get a runtime error, or you get very gibberish outputs

- Nothing can prevent this currently, but try setting `device_map='sequential'` when loading the model or run it on two different kinds of GPUs (e.g., `NVIDIA RTX A6000 or NVIDIA RTX A6000 Ada Generation`)